CHAPTER 1

DESIGNING A VENDING MACHINE

1 Getting Ready: Vending Machine 2
2 Requirements for the Vending Machine 4
3 Use Case Diagram for the Vending Machine 5
4 Class Diagram for the Vending Machine 8
5 Activity Diagram for the Vending Machine 20
6 Code for the Vending Machine 21
7 Summary: Designing a Vending Machine 27
8 Key Insights and Tips: Designing a Vending Machine . 31

In this chapter, you will learn the design of a vending machine, including requirements, use case, class, activity diagrams, and implementation code.

1 Getting Ready: Vending Machine

1.1 Problem definition

A **vending machine** is an automated self-service unit that provides users with products such as snacks, beverages, and chocolates. The machine contains multiple racks with products placed in slots. Users interact with the machine by inserting money, selecting the product they wish to buy, and receiving the selected item if the transaction is valid. The vending machine manages product dispensing based on the selection and payment, handles change, and maintains the state of its inventory.
In this LLD interview case study, you'll focus on:

- Managing the inventory of products using racks and slots within a vending machine.
- Handling user interactions for inserting money, selecting products, and dispensing items.
- Processing payments securely, including calculating and returning change for cash transactions.
- Managing different machine states (idle, accepting money, dispensing).

Note: This vending machine design can be adapted to automated retail kiosks, such as ticketing or electronics dispensers.
The diagram below elaborates on the process of product purchasing using the vending machine:

1.2 Expectations from the interviewee

Although the vending machine problem is a simpler design problem asked in interviews, the interviewer still has some expectations. The following provides an overview of what the interviewer wants to hear you discuss in more detail during the interview.

States of the vending machine

An interviewer would also expect you to discuss the different states of the vending machine. You may ask the following set of questions:

- What function do the vending machines perform? Alternatively, how many different states can the vending machines have?
- After inserting money into the machine, what does the system do?
- Who presses the vending machine button, and what happens after pressing it?
- What does the dispense function do?
- If the vending machine is in a dispense state, is it possible to insert money?

- Would you be able to do so if you are in `NoMoneyInsertedState` and try to select a product without paying money?

Money handling

One of the most significant attributes of the vending machine system is how it receives, calculates, and returns money. You may ask the interviewer the questions listed below:

- What should the system do if we pay less than the product price?
- What should the system do if we pay more than the product price?
- Can the credit card input money, or can only cash be used?

1.3 Design approach

We will design this vending machine system using a bottom-up approach:

- First, we'll identify core entities like `Product`, `Rack`, `Slot`, and `Payment`, and define their responsibilities.
- Next, we'll model inventory and transaction handling, including product management, payment processing, and change return.
- Finally, we'll design the overall `VendingMachine` as a state-driven system, ensuring that our design is modular and extensible by following SOLID principles.

This approach will also consider edge cases, concurrency, and future enhancements such as digital payments or real-time inventory updates. Diagrams and code will be used to illustrate the design in later lessons.

1.4 Design pattern

Discussing the design patterns that a vending machine falls under during an interview is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a vending machine? Kindly elaborate on your choice(s).

Let's explore the requirements of the vending machine problem in the next lesson.

1.5

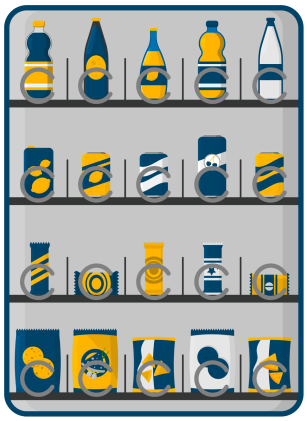
2 Requirements for the Vending Machine

This lesson outlines the functional and operational requirements for the Vending Machine system. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

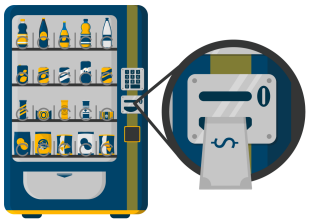
2.1 Requirement collection

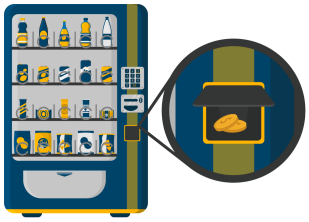
The requirements for the vending machine problem are defined below:

- **R1:** The vending machine stores and manages different types of products, each placed at a unique slot within the machine.
- **R2:** The vending machine can be in one of these three states:
  - **NoMoneyInsertedState:** There is no money inserted into the machine.
  - **MoneyInsertedState:** Money is inserted into the machine.
  - **DispenseState:** The machine gives out the product.
- **R3:** The system has two primary actors: the customer and the operator.
- **R4:** The admin can add or remove new products to the machine.



- **R5:** The system should allow the users to select a product they want to purchase from the machine by specifying the rack number.
- **R6:** Users can insert money (cash) into the machine.
- **R7:** The system should be able to calculate the money inserted into the machine.
- **R8:** The system should check whether the user inserted the exact amount required for the specific product into the machine.
- **R9:** If the amount exceeds the product price, the system should change the user back and dispense the product.
- **R10:** If the amount exceeds the product price, the system should display an error message and return the money.





We've identified our requirements for the problem, and in the next lesson, we will define the class diagram of our vending machine system.

3 Use Case Diagram for the Vending Machine

Let's build the use case diagram for the vending machine system and understand the relationships between its main actors and functions. First , we'll define the different elements of our system, followed by the complete use case diagram.

3.1 System

Our system is the vending machine. It manages automated product selection, payment, and goods dispensing through a self-service interface.

3.2 Actors

Now, we'll define the main actors of our vending machine.

Primary actors

- **Customer:** Can view available products, select products to purchase, insert money, collect dispensed products, and receive change.
- **Operator:** Inherits all customer actions. Additionally, the operator can add new products, remove products, and remove collected cash from the machine.

3.3 Use cases

In this section, we will define the vending machine's use cases. We have listed them according to their respective interactions with a particular actor.

Customer

- **View products:** To view all available products in the vending machine.
- **Select product:** To select a product to buy by specifying the rack or slot number.
- **Insert money:** To insert cash to pay for the selected product.
- **Take product:** To collect the dispensed product from the vending machine.
- **Take change:** To collect change, if any, after the transaction.

Operator

Inherits all c*ustomer* actions, plus:

- **Add product:** To add new products to the vending machine.
- **Remove product:** To remove products from the vending machine.
- **Remove cash:** To remove cash collected from the machine.

System

- **Search product:** To find and verify the selected product's availability in the machine.
- **Validate money:** To ensure the user has inserted enough money to buy the selected product.
- **Dispense product:** To dispense the selected product to the user.
- **Return change:** To return change to the customer if the inserted amount exceeds the product price.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

The operator is a specialized role that extends the customer's capabilities (inherits all customer use cases, plus additional administrative ones).

Associations

The table below shows the association relationship between actors and their use cases.

Customer	**Operator**	**System**
View products	Add product	Search product
Select product	Remove product	Dispense product
Insert money	Remove cash	Validate money
Take product	View product	Return change
Take change	Select product	
	Insert money	
	Take product	
	Take change	

Include

In the use case diagram, "include" relationships show that one use case always incorporates the behavior of another. This helps clarify system workflows and makes the diagram modular and reusable.

- **Select product → Search product**
  - When a customer selects a product, the system must search for the selected product in its inventory to verify availability before proceeding.
  - This ensures the product is in stock and ready for purchase.
- **Validate money → Dispense product**
  - After the customer inserts money and selects a product, the system must validate that the money is legal and sufficient for the purchase.
  - Once validation succeeds, the "Dispense product" use case is included as the next step to physically provide the product to the customer.
- **Insert money → Validate money**
  - Once the customer inserts money to choose a product, the system checks whether the inserted amount is valid for purchasing the product. If it is, the customer proceeds to buy the product; otherwise, the system will ask the user to enter more money.

- **Validate money → Return change**
  - If the inserted amount is greater than the product price, after validating the money, the system must include the behavior of calculating and returning the correct change to the customer.

3.5 Use case diagram

Here's the use case diagram of the vending machine:

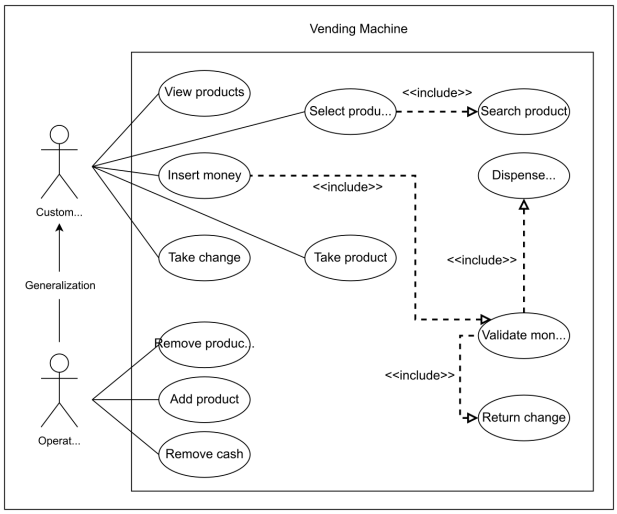


Figure 1.1: The use case diagram of the vending machine

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Vending Machine

In this lesson, we use a bottom-up approach to identify and design the classes and interfaces for our vending machine system. The focus is on clear responsibility separation, applying the State design pattern, and modeling relationships that reflect real-world constraints. Let 's explore each component and how they work together to realize our requirements.

4.1 Components of a vending machine

As mentioned, we should design the vending machine using a bottom-up approach.

State pattern and state

We use the State design pattern to manage the vending machine's changing behavior depending on its current mode. This pattern encapsulates the machine's different operational "states" as separate classes, each handling user actions in its own way.

State interface We define a `State` interface (or abstract base class), which exposes the two main user-facing actions:

- `insertMoney(amount)`: For inserting cash into the machine.
- `selectProduct(rackNumber)`: For selecting a product to purchase.

The machine is in exactly one of the following three concrete states at any moment. Each state provides its logic for these actions:

1. `NoMoneyInsertedState`
   (a) `insertMoney`: Records the inserted amount and transitions the machine to the `MoneyInsertedState`.
   (b) `selectProduct`: Rejects the request with a message (e.g., "Please insert money first").
2. `MoneyInsertedState`
   (a) `insertMoney`: Adds to the current balance.
   (b) `selectProduct`:
      i. Checks if the selected product is available and compares the current balance to the product price:
         A. **Underpaid:** Returns all inserted money and resets to `NoMoneyInsertedState`.
         B. **Exact payment:** Dispenses the selected product and transitions to `DispenseState`.
         C. **Overpayment:** Dispenses the product, returns the change, and transitions to `DispenseState`.
3. `DispenseState`
   (a) `insertMoney`: Does not accept any new actions; prompts the user to wait (e.g., "Please wait, dispensing in progress...").
   (b) `selectProduct`: Same as above; defers all actions while dispensing.
   (c) After the product (and any change) is delivered, the machine resets to `NoMoneyInsertedState`.

Every state implements some functions, as shown below:

R2: The vending machine can be in one of these three states:

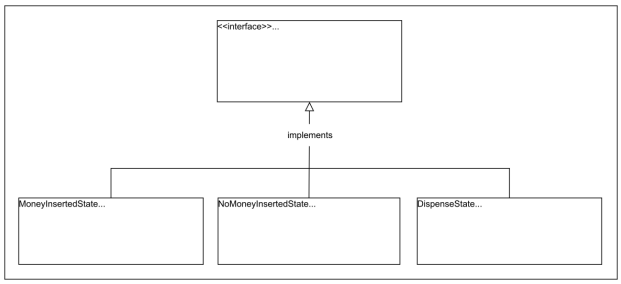


Figure 1.2: The class diagram of the State interface and its subclasses

- There is no money inserted into the machine.
- Money is inserted into the machine.
- The machine gives out the product.

R6: Users can insert money (cash) into the machine; the system should accept, validate, and count the amount.
R7: The system should be able to calculate the money inserted into the machine.
R8: The system should check whether the user inserted the exact amount required for the specific product into the machine.
R9: If the amount exceeds the product price, the system should change back to the user and dispense the product.
R10: If the amount exceeds the product price, the system should display an error message and return the money.

> **Note:** According to the implementation of the State design pattern, all functions will be available in each state. However, not every function must have a meaningful definition for that particular state.

Product

The `Product` class contains the details of a particular product available in the vending machine. Each product has a `name`, `id`, `price`, and its `type` associated with it as presented below:

R1: The vending machine stores and manages different types of products, each placed at a unique slot within the machine.
R5: The system should allow the users to select a product they want to purchase from the machine by specifying the rack number.

Rack

The `Rack` class is used to identify the location of the products in the vending machine. Every rack has a specific `rackNumber` as an identifier, the `product` it has, and the `quantity` of the product. The class representation is provided below:

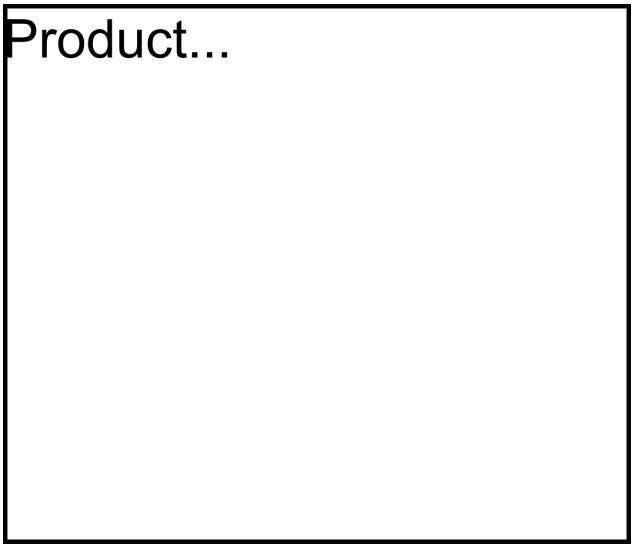


Figure 1.3: The class diagram of the Product class

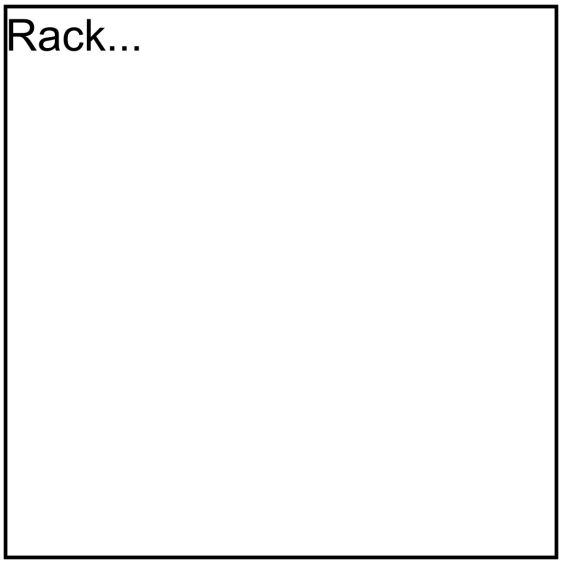


Figure 1.4: The class diagram of the Rack class

R1: The vending machine stores and manages different types of products, each placed at a unique slot within the machine.
R5: The system should allow the users to select a product they want to purchase from the machine by specifying the rack number.

Inventory

The `Inventory` class manages a collection of racks inside the vending machine. Internally , it keeps a map from each rack number to its corresponding `Rack` object. It provides three core operations:

1. addRack(Rack rack): Register a new rack in the machine.
2. getRack(int rackNumber): Look up and return the rack for the given position (or `null` if none exists).
3. `allRacks()`: Retrieve every rack currently in the machine.

This design makes it easy to load, dispense, and inspect products by rack number without exposing any lower-level list or count fields.
The class representation is provided below:

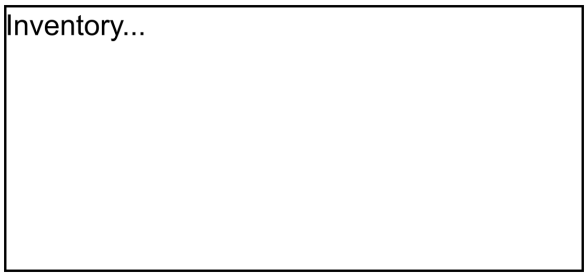


Figure 1.5: The class diagram of the Inventory class

R1: The vending machine stores and manages different types of products, each placed at a unique slot within the machine.
R4: The admin can add or remove new products to the machine.

VendingMachine

The `VendingMachine` class is the central context in our State pattern design. It is a Singleton (only one instance exists) and holds:

- Current state (`State`)
- Current balance (the total cash inserted so far)
- Inventory (the collection of racks and their contents)

All external calls—`insertMoney(amount)` and `selectProduct(rackNumber)`—simply delegate to whatever `State` object is active. The machine itself handles:

1. State transitions (No money → Money inserted → Dispense → No money)
2. Actual dispensing/refunding logic (once the state says it's "OK" to dispense)
3. Admin operations (loading racks, collecting cash)—kept entirely outside the State interface.

By pushing only the two user actions into the State subclasses, and keeping everything else in the context, we get a clear separation of "what changes when" vs. "how things actually happen." The class representation is provided below:

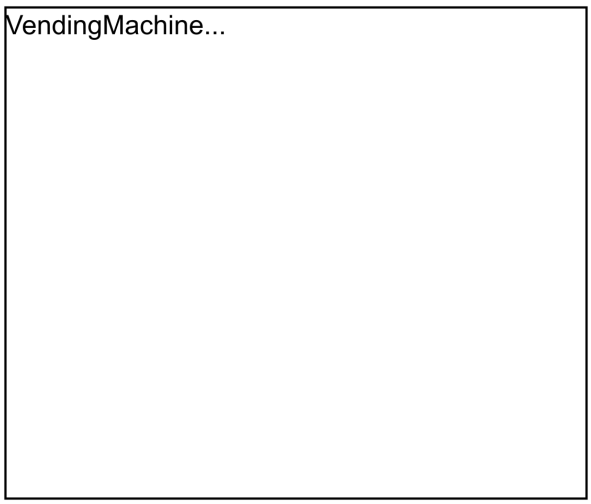


Figure 1.6: The class diagram of the VendingMachine class

R2: The vending machine can be in one of these three states:

- **NoMoneyInsertedState:** There is no money inserted into the machine.
- **MoneyInsertedState:** Money is inserted into the machine.
- **DispenseState:** The machine gives out the product.

R5: The system should allow the users to select a product they want to purchase from the machine by specifying the rack number.
R6: Users can insert money (cash) into the machine.
R7: The system should be able to calculate the money inserted into the machine.
R8: The system should check whether the user has inserted the exact amount required for the specific product into the machine.
R9: If the amount exceeds the product price, the system should change the user back and dispense the product.
R10: If the amount exceeds the product price, the system should display an error message and return the money.

Enumeration

The enumeration required to design the vending machine system is provided below:

ProductType We need to create an enumeration to keep track of the type of product, whether it is chocolate, a snack, a beverage, or another type.

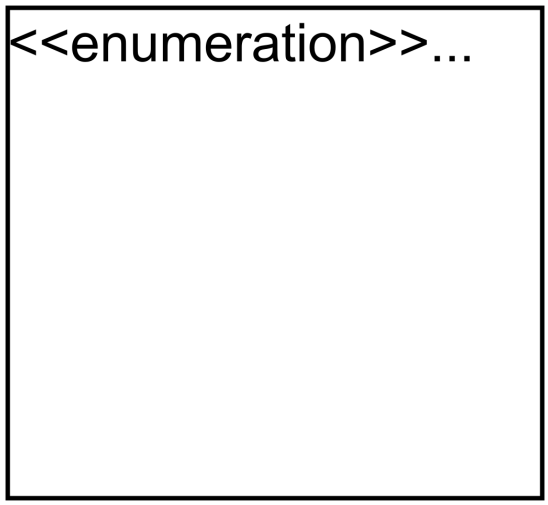


Figure 1.7: The ProductType enumeration

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our vending machine.

Composition

We use composition to show strong "owns-a" relationships wherever one object cannot meaningfully exist without its container:

1. `VendingMachine` is composed of `Inventory`.
   (a) The machine contains exactly one `Inventory` instance.
   (b) If the machine is torn down, its inventory goes with it.
2. `Inventory` is composed of `Rack`.
   (a) The inventory manages a collection of racks (one per slot).
   (b) Each `Rack` lives and dies inside its `Inventory`.

3. `Rack` is composed of `Product`.

   (a) A rack always holds a specific `Product` (plus a quantity).

   (b) You cannot have a rack without knowing what product it contains.

Visually, each arrow is a filled diamond at the container end:

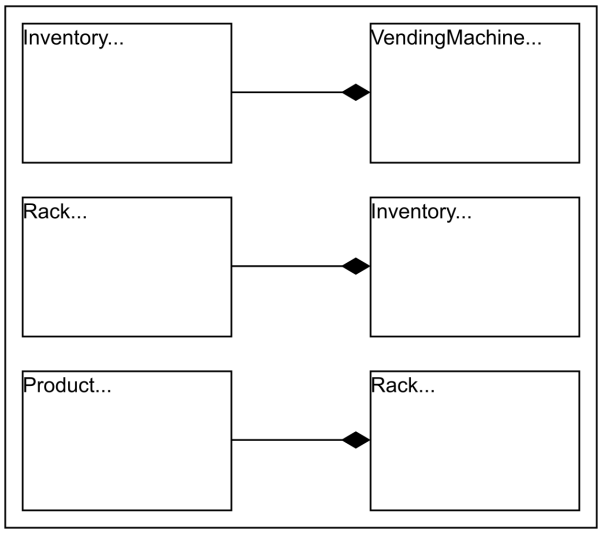


Figure 1.8: The composition relationship between classes

This structure ensures that:

- Only the `VendingMachine` can create or destroy the entire `Inventory`.
- Only `Inventory` can add or remove `Rack` objects.
- Only each `Rack` can load or unload its own `Product`.

By modeling these as compositions, we clearly express ownership and the life cycle dependencies among our core classes.

Aggregation

The class diagram has the following aggregation relationships:

- The `VendingMachine` class contains the `State` interface.

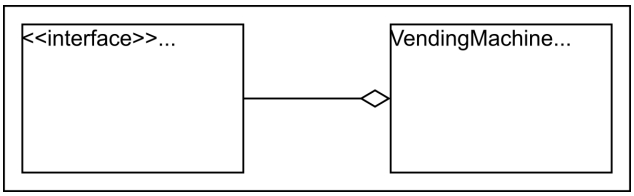


Figure 1.9: The aggregation relationship between classes

Association

Besides the strong ownership shown by our composition arrows, we also have simple "uses-a" links between classes. In UML, these are unfilled-arrow or line associations with no diamond:

- `VendingMachine` → `Inventory`
  The machine holds one `Inventory` object and calls its lookup methods to find racks.
- `VendingMachine` → `State`
  The machine keeps a reference to its current `State` (No money, Money inserted, or Dispense) and delegates `insertMoney`/`selectProduct` to it.
- `Inventory` → `Rack`
  Internally, `Inventory` stores many `Rack` instances (one per slot) and looks them up by rack number.
- `Rack` → `Product`
  Each rack refers to exactly one `Product` (plus a quantity) when loaded.

Visually, you'd draw each of these as a straight line (or arrow) from the "user" class to the "used" class, with multiplicities as needed.

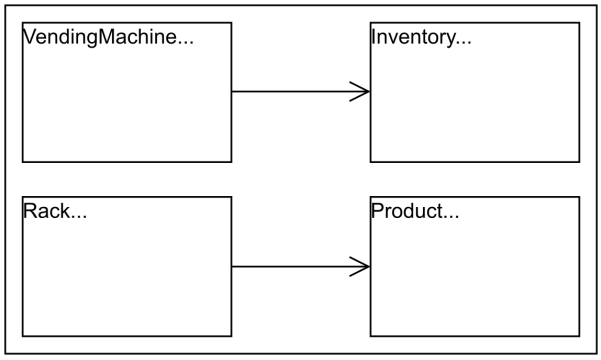


Figure 1.10: The association relationship between classes

Inheritance

Inheritance in object-oriented design allows classes to share contracts (interfaces/abstract classes) or behaviors (concrete classes). Our three concrete state classes—`NoMoneyInsertedState`,

`MoneyInsertedState`, and `DispenseState`—all implement the common `State` interface. Referring to the UML under the "State pattern and State" section, this is shown with a solid line and a hollow triangle pointing to the superclass or the `State` interface.

- `State` defines the two key operations (`insertMoney` and `selectProduct`) without prescribing how they work.

- Each concrete state class inherits that contract and provides its own behavior for those methods, according to the vending machine's current mode.

This arrangement lets the `VendingMachine` treat all states uniformly (polymorphism), while each subclass specializes the response to user actions exactly as its particular state requires.

> **Note:** We have already discussed this inheritance relationship between classes in the component section above.

4.3 Class diagram for the vending machine

This section outlines the multiplicity (cardinality) relationships between the main classes in our Vending Machine system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	Target	Multiplicity	Reason
VendingMachine	Inventory	1 — 1	Each VendingMachine has exactly one Inventory, which manages all racks and products. The inventory is tightly coupled and cannot exist without its machine.
Inventory	Rack	1 — 1..*	Each Inventory manages one or more Rack objects. A machine must have at least one rack to function, and can have many racks for different products.
Rack	Product	1 — 1	Each Rack holds exactly one Product at a time (plus quantity) and is designed for one product type throughout its life cycle.
VendingMachine	State	1 — 1 (current), 1 — 3 (total)	The VendingMachine references exactly one current State at runtime, but owns all three possible state instances internally (State Pattern).
State	Concrete State (NoMoneyInsertedState, MoneyInsertedState, DispenseState)	1 — 3 (implements)	The State interface is implemented by three concrete classes, representing the possible operational states of the vending machine.

Here is the complete class diagram for our vending machine:

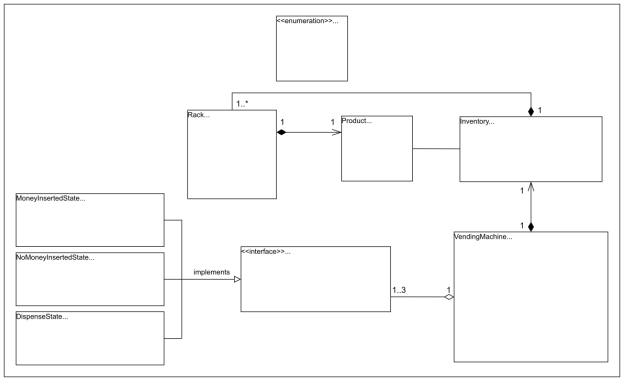


Figure 1.11: The class diagram of the vending machine

4.4 Design pattern

We apply the *State pattern* because the vending machine's behavior changes depending on its current mode. At runtime, it lives in exactly one of three states:

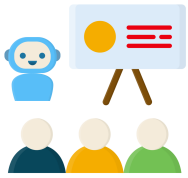
- "No money" state
- "Money inserted" state
- "Dispense" state

Each state exposes the same two operations (`insertMoney` and `selectProduct`), but implements them differently—rejecting selections when no cash is in, accumulating balance when money is inserted, validating price vs. balance when a product is chosen, or blocking all input while dispensing.

> By encapsulating each behavior in its own state class, we avoid large `if/else` or `switch` statements inside the machine, making the code easier to maintain and extend (e.g., adding a "Maintenance" or "OutOfOrder" state in the future without touching existing logic).

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the vending machine, or they can ask some follow-up questions. Let 's see some examples of additional requirements:
Refund/Cancel: The vending machine should have the option to cancel the operation. In that case, the customers will get a refund. The class diagram provided below shows the refund functionality in all states:

In the class diagram above, we can see a function, refundFullMoney(), in all states, which is used to refund the full money. Here is the definition of the `refundFullMoney()` function according to each state:

- `NoMoneyInsertedState`: In this state, the refundFullMoney() function throws an exception or warning, "Please insert some cash first," because we have not inserted any money yet.
- `MoneyInsertedState`: In this state, we know the customer inserted the cash because we shifted the vending machine's state from `NoMoneyInsertedState` to `MoneyInsertedState`. If the customer uses the refund option, the `refundFullMoney()` function will be called and return the full amount to the change tray.

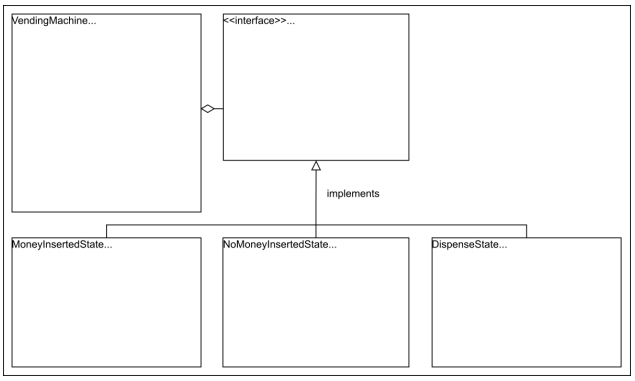


Figure 1.12: Modified class diagram of the vending machine

- `DispenseState`: In this state, the `refundFullMoney()` function is blocked. If the customer uses the refund option, the system does nothing. This is because the customer has selected a product, and the vending machine is in the dispense state.

We have completed the class diagram of a vending machine according to the requirements. In the next lesson, let's design the activity diagram of the vending machine.

5 Activity Diagram for the Vending Machine

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. We can create different activity diagrams for our vending machine. In this lesson, we will create an activity diagram for purchasing a product from the vending machine.

5.1 Product purchase from the vending machine

The following are the states and actions involved in this activity diagram.

States

Initial state: The customer inserts money into the vending machine.
Final state: The customer receives the desired product.

Actions

The customer interacts with the vending machine and inserts money. The customer selects their desired product. If that product is available, they get the product and the change. Otherwise , the system shows a message of product unavailability and asks the customer to select another product.
Based on the order above, the activity diagram of a product purchase from the vending machine is shown below:

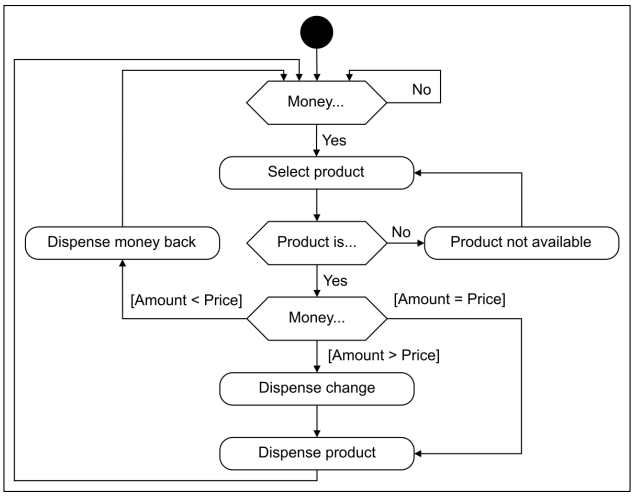


Figure 1.13: The activity diagram for the product purchase from the vending machine

We've looked at the activity diagram of our vending machine. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

6 Code for the Vending Machine

We've reviewed different aspects of the vending machine problem and observed the attributes attached to the problem using various UML diagrams. Let us now explore the more practical side of things, where we will work on implementing the vending machine using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the vending machine:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Vending Machine system.

6.1 Vending machine classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

Note: For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations

The following code defines the enumeration used in the vending machine system:

Note: JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
public enum ProductType {
    CHOCOLATE, SNACK, BEVERAGE, OTHER
}
```

State

The `State` interface declares the two user-facing operations whose behavior varies by the machine's current mode. Each concrete state class implements these methods, but leaves their bodies empty here as placeholders for the real logic in `VendingMachine`.

The State interface and its derived classes

```
// State.java
public interface State {
    /** Called when the user inserts money. */
    void insertMoney(VendingMachine vm, double amount);

    /** Called when the user selects a rack. */
    void selectProduct(VendingMachine vm, int rackNumber);
}

// NoMoneyInsertedState.java
public class NoMoneyInsertedState implements State {
    @Override
    public void insertMoney(VendingMachine vm, double amount) {
        // to be implemented: record amount and switch to MoneyInsertedState
    }

    @Override
    public void selectProduct(VendingMachine vm, int rackNumber) {
        // to be implemented: reject selection when no money
    }
}

// MoneyInsertedState.java
public class MoneyInsertedState implements State {
```

```
    @Override
    public void insertMoney(VendingMachine vm, double amount) {
        // to be implemented: add to current balance
    }

    @Override
    public void selectProduct(VendingMachine vm, int rackNumber) {
        // to be implemented: check availability and payment, then dispense or
            refund
    }
}

// DispenseState.java
public class DispenseState implements State {
    @Override
    public void insertMoney(VendingMachine vm, double amount) {
        // to be implemented: defer until dispensing completes
    }

    @Override
    public void selectProduct(VendingMachine vm, int rackNumber) {
        // to be implemented: defer until dispensing completes
    }
}
```

Product, rack, and inventory

At the heart of our vending-machine model are three classes that work together to represent what's in each slot and where:

- `Product` encapsulates the details of an item for sale—its unique ID, display name, price, and category (chocolate, snack, beverage, or other).
- `Rack` represents a single slot in the machine. It knows which `Product` it holds and how many units remain. When you load or dispense stock, you do so by interacting with the appropriate rack.
- `Inventory` serves as the lookup service for every rack. It maintains a map from rack numbers to `Rack` instances, so when the machine needs to check availability or list its contents, it simply asks the `Inventory` for rack # N (or for a list of all racks).

Together, these classes cleanly separate "what the items are" (`Product`), "where they live" (`Rack`), and "how to find them" (`Inventory`).
The definition of these classes is provided below:

The Product, Rack, and Inventory classes

```
// ===== Product =====
public class Product {
    private final String name;
```

```java
    private final int id;
    private final double price;
    private final ProductType type;

    public Product(int id, String name, double price, ProductType type) {
        // constructor implementation here
    }

    public int getId() { return 0; }
    public String getName() { return null; }
    public double getPrice() { return 0.0; }
    public ProductType getType() { return null; }
}

// ===== Rack =====
public class Rack {
    private final int rackNumber;
    private Product product;
    private int quantity;

    public Rack(int rackNumber) {
        // constructor implementation here
    }

    public int getRackNumber() { return 0; }
    public boolean isEmpty() { return false; }
    public void loadProduct(Product product, int qty) { }
    public Product peekProduct() { return null; }
    public void dispenseOne() { }
}

// ===== Inventory =====
import java.util.Collection;

public class Inventory {
    public void addRack(Rack rack) { }
    public Rack getRack(int rackNumber) { return null; }
    public Collection<Rack> allRacks() { return null; }
}
```

Vending machine

The `VendingMachine` class is the central controller of our system. Implemented as a Singleton, it encapsulates:

- It creates three State instances (no money, money inserted, dispense) up front, between which it switches.
- `currentState`: The active state object.
- `currentAmount`: Running total of cash the user has put in.
- `selectedRack`: The rack number the user has picked (–1 if none).

- `inventory`: The Inventory instance that owns all rack objects.

All user calls (`insertMoney` and `selectProduct`) are simply delegated to the `currentState`. The machine handles the actual dispensing/refunding and resets to the "No money" state when each transaction completes. Administrative methods (`addRack`/`loadProduct`) sit on the context here, not in the State interface.
The definition of this class is given below:

The VendingMachine class

```
public class VendingMachine {
    private final State noMoneyState = new NoMoneyInsertedState();
    private final State moneyInsertedState = new MoneyInsertedState();
    private final State dispenseState = new DispenseState();

    private State currentState = noMoneyState;
    private double currentAmount = 0.0;
    private int selectedRack = -1;
    private final Inventory inventory = new Inventory();

    private static VendingMachine instance;
    private VendingMachine() {}

    public static VendingMachine getInstance() {
        if (instance == null) {
            instance = new VendingMachine();
        }
        return instance;
    }

    State getNoMoneyState() { return noMoneyState; }
    State getMoneyInsertedState() { return moneyInsertedState; }
    State getDispenseState() { return dispenseState; }

    void setState(State state) { this.currentState = state; }

    double getCurrentAmount() { return currentAmount; }
    void addToCurrentAmount(double amt) { /* to be implemented */ }
    Inventory getInventory() { return inventory; }
    void setSelectedRack(int rack) { /* to be implemented */ }

    public void insertMoney(double amount) {
        currentState.insertMoney(this, amount);
    }

    public void selectProduct(int rackNumber) {
        currentState.selectProduct(this, rackNumber);
    }

    public void dispenseProduct() { /* to be implemented */ }
    public void refund() { /* to be implemented */ }
    public void addRack(Rack rack) { /* to be implemented */ }
    public void loadProduct(int rackNumber, Product product, int qty) { /* to be
        implemented */ }
```

```
    public void showInventory() { /* to be implemented */ }
}
```

6.2 Executable code: Vending machine

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Vending Machine system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Sets up the vending machine with racks, products, and their quantities.
- **Scenario 1 (Customer buys a product):** The customer inserts money, selects a product using the rack number, and either receives the product and change (if any) or an error message (if payment is insufficient).
- **Scenario 2 (Operator adds/removes a product):** The operator restocks or removes a product from the machine.
- **Scenario 3 (Handling edge cases):** Demonstrates behaviors such as underpayment, exact payment, and overpayment, as well as selecting an empty rack.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Vending Machine

```
public class Driver {
    public static void main(String[] args) {
        VendingMachine vm = VendingMachine.getInstance();

        // Setup
        vm.addRack(new Rack(1)); vm.addRack(new Rack(2)); vm.addRack(new Rack(3));
        Product choc = new Product(101, "Chocolate Bar", 1.50, ProductType.CHOCOLATE
            );
        Product snack= new Product(102, "Potato Chips", 2.00, ProductType.SNACK);
        Product bev = new Product(103, "Soda Can", 2.50, ProductType.BEVERAGE);
        vm.loadProduct(1, choc, 5);
        vm.loadProduct(2, snack, 3);
        vm.loadProduct(3, bev, 2);
        vm.showInventory();

        // Scenario 1: Exact payment
        System.out.println("========== Scenario 1: Exact Payment ==========");
        vm.insertMoney(1.50);
```

```
        vm.selectProduct(1);

        // Scenario 2: Overpayment & Change
        System.out.println("========== Scenario 2: Overpayment & Change ==========")
            ;
        vm.insertMoney(3.00);
        vm.selectProduct(2);

        // Scenario 3: Underpayment & Refund
        System.out.println("========== Scenario 3: Underpayment & Refund =========="
            );
        vm.insertMoney(1.00);
        vm.selectProduct(3);

        // Scenario 4: Deplete Rack & Retry
        System.out.println("========== Scenario 4: Deplete Rack 3 & Retry ==========
            ");
        vm.insertMoney(5.00); vm.selectProduct(3);
        vm.insertMoney(2.50); vm.selectProduct(3);
        vm.insertMoney(2.50); vm.selectProduct(3);

        vm.showInventory();
    }
}
```

6.3 Wrapping up

In this chapter, we've explored the complete design of a vending machine. We 've also examined how a basic vending machine can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

7 Summary: Designing a Vending Machine

Now that you've completed the vending machine case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

7.1 Key requirements

This section outlines the primary functional requirements that shaped the design of the vending machine system.

1. The machine must store and manage various products, assigning each item to a unique slot.

2. The system operates in three distinct states: `NoMoneyInsertedState`, `MoneyInsertedState`, and `DispenseState`.

3. The system has two main actors: a Customer, who buys products, and an Operator, who manages the machine.

4. An Operator must be able to add or remove products from the machine's inventory.

5. Users must be able to select a product by specifying its rack number.

6. Users can insert cash as payment.

7. The system must accurately calculate the total amount of money inserted.

8. The system must verify if the inserted amount is sufficient for the selected product.

9. If the user pays more than the product price, the system must dispense the product and return the correct change.

10. If the user pays less than the product price, the system must display an error and return the inserted money.

7.2 System actors

The following actors interact with the vending machine system:

- **Customer:** A primary actor who can view and select products, insert money, and collect their purchased item and any change.

- **Operator:** A specialized actor with all the capabilities of a Customer, but can also perform administrative tasks like adding/removing products and collecting cash from the machine.

7.3 Important classes and relationships

The design is built around several core classes that model the system's entities and their interactions. The table below summarizes these classes and their most important relationships.

Class	Description	Important Relationships
VendingMachine	The central singleton class that manages the current state, balance, and inventory. It orchestrates the overall workflow.	Composition with Inventory; Association with State.
State	An interface defining the core user actions (insertMoney, selectProduct).	Implemented by NoMoneyInsertedState, MoneyInsertedState, and DispenseState.
NoMoneyInsertedState	A concrete state where the machine is idle and waiting for money. Only accepts money insertion.	Implements the State interface.
MoneyInsertedState	A concrete state activated after money is inserted. Handles product selection and payment validation.	Implements the State interface.
DispenseState	A concrete state responsible for dispensing the product and change, then resetting the machine.	Implements the State interface.
Inventory	Manages the collection of all racks within the machine.	Composition with Rack.
Rack	Represents a physical rack in the machine that holds a specific product and its quantity.	Composition with Product.
Product	A data class storing an item's name, price, and type.	Associated with a Rack.

7.4 Design highlights

This section covers the key design decisions, principles, and patterns applied to build a robust and maintainable system.

Design approach

The system was designed using a bottom-up approach. We started by identifying the simplest core entities like, `Product`, and `Rack`, then built upon them to create more complex components, like `Inventory` and the state-driven `VendingMachine` itself. This approach ensures that fundamental components are well-defined before being integrated into the larger system.

Design principles

The design adheres to the five SOLID principles to ensure it is robust, maintainable, and scalable.

- **Single Responsibility principle (SRP):** Each class has a well-defined purpose. For example, the `Inventory` class manages racks, while the `State` classes handle state-specific behaviors.
- **Open/Closed principle (OCP):** The system is open for extension but closed for modification. Using the State pattern, we can introduce new states (e.g., a `MaintenanceState`) without altering the existing `VendingMachine` or state classes.
- **Liskov Substitution principle (LSP):** All concrete state classes (`NoMoneyInsertedState`, `MoneyInsertedState`, `DispenseState`) are substitutable for the `State` interface. The `VendingMachine` interacts with them through the common interface, ensuring consistent behavior.
- **Interface Segregation principle (ISP):** The `State` interface is lean and specific to user actions. Administrative functions like `addRack` are kept within the `VendingMachine` class, so state classes are not forced to implement methods they don't need.
- **Dependency Inversion principle (DIP):** The `VendingMachine` class depends on the `State` interface (an abstraction) rather than its concrete implementations. This decouples the main class from the details of state-specific logic.

Design patterns

The design leverages established design patterns to solve common problems effectively.

- **State pattern:** This pattern is central to the design. It encapsulates the vending machine's varying behavior into separate state objects (`NoMoneyInsertedState`, `MoneyInsertedState`, `DispenseState`), allowing it to alter its behavior when its internal state changes. This avoids complex conditional logic within the main `VendingMachine` class.

7.5 Object interactions

This section describes how key objects collaborate to fulfill user requests, focusing on the flow of method calls between them.

- **Money insertion:** When a `Customer` inserts money, the `VendingMachine` delegates the call to its current state object. If the state is `NoMoneyInsertedState`, it accepts the money, updates the machine's balance, and transitions the machine's state to `MoneyInsertedState`.
- **Product selection:** A `Customer` selects a product by its rack number. The `VendingMachine` passes this request to the current `MoneyInsertedState` object. The state object then collaborates with the `Inventory` to check the product's availability and price.
- **Payment validation and dispensing:** The `MoneyInsertedState` object compares the machine's current balance to the product's price.

  - If payment is sufficient, it instructs the `VendingMachine` to dispense the product and any required change. The machine's state then transitions to `DispenseState`.
  - If payment is insufficient, it instructs the `VendingMachine` to refund the money and transitions the state back to `NoMoneyInsertedState`.

- **Dispensing and reset:** Once in `DispenseState`, the machine dispenses the item and change. After the process, the state automatically returns to `NoMoneyInsertedState` to prepare for the next transaction.

7.6 System workflow

This section outlines the primary end-to-end scenario of a customer purchasing a product, as visualized in the activity diagram.

- **Initial state:** The process begins with the machine idle, waiting for a user to insert money.
- **Money insertion:** A customer inserts cash into the machine.
- **Product selection:** The customer selects a desired product. The system then checks if the selected product is available in the inventory. If not, it informs the customer and prompts them to make another selection.
- **Payment validation:** If the product is available, the system validates the payment.
  - If the inserted amount exceeds the product's price, the machine dispenses the money back to the customer, and the flow returns to the money insertion step.
  - If the amount is exactly the price, the system dispenses the product.
  - If the amount exceeds the price, the system first dispenses the correct change and then dispenses the product.
- **Final state:** The customer receives their product (and any change), concluding the transaction. The machine then resets for the next customer.

8 Key Insights and Tips: Designing a Vending Machine

You've completed the "Designing a Vending Machine" case study. This lesson builds on that experience by exploring common design pitfalls and providing targeted guidance for interview scenarios. To help reinforce your understanding, a short quiz and a list of related case studies focus on similar design principles.

8.1 Common mistakes

Avoiding these common mistakes will help you develop a more efficient and scalable vending machine system:

- **Not using the State pattern:** Failing to manage the distinct states of the vending machine (e.g., `NoMoneyInserted`, `MoneyInserted`, `Dispense`) leads to complex and hard-to-maintain conditional logic. You should implement the State design pattern to encapsulate state-specific behaviors in separate classes, making the system more modular and extensible.

- **Hardcoding product information:** Directly embedding product details like price and name into the main `VendingMachine` class makes the system inflexible. You should create a separate Product class and an inventory system to manage items dynamically.

- **Ignoring the Singleton pattern for the vending machine:** Creating multiple instances of the `VendingMachine` class can lead to inconsistent states and resource management issues. You should implement the Singleton pattern to ensure only one instance of the vending machine exists throughout the application.

- **Lack of a clear inventory management system:** Without a dedicated inventory component, tracking product quantities or knowing when an item is out of stock is impossible. You should design an `Inventory` class that maps products to racks and manages their counts.

- **Overlooking the physical rack or slot mechanism:** Abstracting the design too much without considering how products are physically stored and dispensed can be a red flag. You should include a `Rack` or `Slot` class with a product and its quantity to represent the physical layout.

8.2 Interview tips

Here are some essential tips to help you prepare for an interview where you're asked to design a vending machine:

Tip	Explanation
Clarify the requirements early.	Ask questions to define the scope, such as "What forms of payment are accepted (cash, card)?", "Does the machine need to give change?", and "Who will be restocking the machine?". This demonstrates a methodical approach to problem-solving and ensures you're designing the right system.
Break down the components logically.	Discuss the design regarding its core components, such as Product, Inventory, Rack, and the various State classes. This shows that you can think object-orientedly and effectively separate concerns.
Discuss money handling in detail.	Explain the logic for calculating the total inserted amount, comparing it to the product price, and returning the correct change. Detailing this shows thoroughness and attention to core workflows.
Walk through a purchase scenario.	Verbally trace the user's steps to buy an item, explaining how the system transitions between states. Example: "The machine starts in NoMoneyInsertedState. After the user inserts money, it transitions to MoneyInsertedState for product selection."
Mention extensibility.	Briefly discuss supporting future extensions—e.g., adding credit card payments via a new payment strategy, or introducing new product types without modifying core logic.

8.3 Test your understanding

Take a short quiz to evaluate your understanding of the vending machine design.

Designing a Vending Machine

Question 1: Which design pattern is most suitable for managing the different phases of a vending machine's operation (e.g., idle, accepting money, dispensing)?

- Observer pattern
- **[CORRECT]** State pattern
  Explanation: The State pattern is ideal because the machine's behavior changes based on its internal state, and this pattern encapsulates those behaviors into distinct state objects.
- Singleton pattern
- Factory pattern

Question 2: What is the primary responsibility of the `Inventory` class?

- To calculate the change to be returned to the user.
- To manage the current state of the vending machine.
- To process payment from the user.
- **[CORRECT]** To hold the collection of products and their locations (racks).
  Explanation: The `Inventory` class is designed to keep track of which products are in which racks and their available quantities.

Question 3: The relationship between the `Inventory` and `Rack` classes is what type of association?

- Aggregation
- Inheritance
- **[CORRECT]** Composition
  Explanation: It's a composition relationship because `Rack` s are integral parts of the `Inventory` and are managed within it; they don't exist independently.
- Implementation

Question 4: Why is the `VendingMachine` class suggested to be a Singleton?

- To ensure it can be easily extended.
- **[CORRECT]** To ensure only one instance of the machine exists, manage a single state and inventory.
  Explanation: A Singleton pattern ensures that there is only one globally accessible point for the vending machine, preventing conflicts and inconsistent states.
- To allow for multiple users to access it simultaneously.
- To create different types of vending machines.

Question 5: What is the relationship between the `NoMoneyInsertedState`, `MoneyInsertedState`, and `State` components?

- Composition
- Aggregation

- Association
- **[CORRECT]** Inheritance/Implementation
  Explanation: The concrete state classes (`NoMoneyInsertedState`, etc.) implement the `State` interface, providing specific behaviors for each state.

8.4 Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the vending machine:

- **Designing an ATM:** This system relies heavily on the State design pattern to manage user sessions (e.g., `Idle`, `CardInserted`, `PinEntered`, `TransactionComplete`) and involves secure transaction processing.
- **Designing an elevator system:** This case study is another classic example of using the State pattern to manage the elevator's movement (e.g., `MovingUp`, `MovingDown`, `Idle`, `DoorsOpen`). It also requires careful management of requests and sequencing.
- **Designing a jigsaw puzzle:** This case study focuses on object composition and managing the state and position of many individual pieces (`PuzzlePiece`) that form a larger whole (`Puzzle`), similar to how products and racks form the inventory.
- **Designing a traffic light system:** This is a simple yet powerful example of a finite state machine. The system transitions between states (`Red`, `Yellow`, `Green`) in a predefined sequence, a core concept in the vending machine design.
- **Designing a simple coffee machine:** This requires managing different states (e.g., `Ready`, `Brewing`, `Dispensing`) and inventory (water, coffee beans, milk), which is similar to the vending machine problem.